

Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Unidade Curricular de Agentes e Sistemas Multiagente

Ano Letivo de 2025/2026

Sistema Multiagente para controlo de tráfego de drones



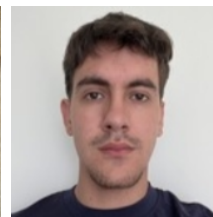
Gustavo Barros
PG61527



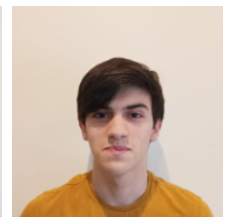
Rui Rodrigues
PG61544



Enzo Vieira
PG61518



João Macedo
PG60274



Jorge Pereira
PG60276

January 14, 2026

ASMA

Índice Geral

1. Introdução	4
2. Caso de Estudo	4
2.1. Objetivos Principais do Programa	5
3. Arquitetura do Sistema Multiagente	6
3.1. Definição dos Agentes	6
3.2. Arquitetura do Sistema	6
3.2.1. Diagrama de Classes	6
3.2.2. Diagrama de Sequência: Compra de Drone	7
3.2.3. Diagrama de Sequência: Compra de Item	8
3.2.4. Diagrama de Sequência: Processo de Negociação	8
4. Comunicação e Protocolos	10
4.1. Protocolos de Comunicação	10
4.1.1. Estrutura Base de Comunicação	10
4.1.2. Tipos de Pacotes	10
4.1.2.1. Protocolo de Compra e Inventário	11
4.1.2.2. Protocolo de Entrega	11
4.1.2.3. Protocolo de Gestão de Drones	11
4.1.2.4. Protocolo de Empréstimo de Drones	12
5. Aspetos adicionais do sistema	12
5.1. Drone	12
5.2. Item	13
5.3. Encomenda	13
5.3.1. Coordenadas	13
5.3.2. Bateria	13
5.3.3. Tempo de entrega	14
5.4. Frontend	14
5.4.1. Cliente	14
5.4.2. Empresa	15
5.4.3. Central	15
6. Análise de Resultados e Crítica	16
6.1. Sumário de Resultados	16
6.2. Análise Crítica	16
7. Recomendações e Trabalho Futuro	17
7.1. Ambiente e Contexto	17
7.2. Gestão e Logística de Inventário	17
8. Conclusão	17
8.1. Reflexão Final	18

Índice de Figuras

Figura 1 Diagrama de Classes	7
Figura 2 Diagrama de Sequência: Compra de Drone	7
Figura 3 Diagrama de Sequência: Compra de Item	8
Figura 4 Diagrama de Sequência: Processo de Negociação	9

Índice de Tabelas

Tabela 1	Mensagens do Protocolo de Compra e Inventário	11
Tabela 2	Mensagens do Protocolo de Entrega	11
Tabela 3	Mensagens do Protocolo de Gestão de Drones	11
Tabela 4	Mensagens do Protocolo de Empréstimo de Drones	12

1. Introdução

A utilização de drones, tem registado um crescimento exponencial nos últimos anos, abrangendo áreas que vão desde a logística e entrega de bens até à monitorização agrícola e operações de procura e salvamento. À medida que o espaço aéreo se torna mais densamente povoado por estas unidades, a gestão manual ou centralizada torna-se ineficiente e propensa a falhas, surgindo a necessidade premente de sistemas de controlo de tráfego autónomos e inteligentes.

Este projeto propõe o desenvolvimento de um **Sistema Multiagente (SMA)** para a gestão e controlo de tráfego de drones, utilizando a biblioteca **SPADE** (Smart Python Agent Development Environment) em linguagem Python. A escolha de uma abordagem baseada em agentes justifica-se pela natureza intrinsecamente distribuída e dinâmica do problema, onde cada drone pode ser modelado como uma entidade autónoma com objetivos próprios e capacidades de interação social.

O sistema desenvolvido assenta nos seguintes objetivos fundamentais:

- Implementar uma arquitetura distribuída onde a coordenação é alcançada através da comunicação entre agentes.
- Adotar protocolos de comunicação normalizados garantindo a interoperabilidade e clareza nas interações.
- Priorizar a troca de informação estruturada através de objetos, permitindo uma manipulação de dados mais complexa e robusta do que a simples troca de texto.
- Explorar mecanismos de coordenação que permitam a coexistência segura de múltiplos drones no mesmo espaço aéreo.

Ao longo deste relatório, será detalhada a modelação do sistema recorrendo a metodologias de **Agent UML**, a implementação técnica dos comportamentos dos agentes e uma análise crítica dos resultados obtidos na simulação do tráfego aéreo.

2. Caso de Estudo

O presente trabalho tem como caso de estudo o desenvolvimento de um sistema de multiagentes inteligentes aplicado a um cenário de comércio eletrónico com entregas automatizadas por drones. O sistema simula um ecossistema no qual existem vários tipos de agentes, nomeadamente clientes, empresas, uma central de gestão de drones e os próprios drones, que interagem entre si de forma autónoma e coordenada.

Os clientes têm como principal função aceder aos sites das empresas, consultar os produtos disponíveis e efetuar encomendas. As empresas, por sua vez, são responsáveis pela gestão dos seus produtos e pela realização das entregas aos clientes. Para tal, necessitam de recorrer a drones, que podem ser adquiridos à central de drones. Esta central atua como entidade intermediária, responsável pela gestão global da frota de drones e pela sua distribuição pelas empresas conforme as necessidades existentes.

Para além disso, os drones existentes no sistema não são homogêneos, possuindo características específicas que influenciam o seu desempenho, tais como a velocidade de deslocação e a capacidade de carga. Estas características tornam cada drone mais adequado a determinados tipos de entrega, dependendo da distância a percorrer e do peso da encomenda. Assim, as empresas não podem escolher os drones de forma aleatória, sendo necessário selecionar aqueles que melhor se adequam às suas necessidades operacionais em cada momento.

Um dos aspetos mais relevantes do sistema ocorre quando uma empresa necessita de efetuar entregas mas não possui drones disponíveis e a central também não tem drones livres para fornecer. Nessa situação, é iniciado um processo de negociação entre empresas, no qual estas podem emprestar drones umas às outras. Este mecanismo de negociação permite a cooperação entre agentes, promovendo uma utilização mais eficiente dos recursos disponíveis e assegurando a continuidade do serviço de entregas. Assim, o sistema demonstra comportamentos típicos de sistemas multiagente, como autonomia, comunicação, cooperação e tomada de decisão distribuída.

2.1. Objetivos Principais do Programa

O principal objetivo deste programa é modelar e implementar um sistema de multiagentes inteligentes que simule um ambiente realista de comércio eletrónico e logística de entregas através de drones, evidenciando a interação e cooperação entre diferentes agentes autónomos.

De forma mais específica, os objetivos do programa são:

- Demonstrar o funcionamento de um sistema multiagente, evidenciando conceitos como autonomia, comunicação, negociação e cooperação entre agentes.
- Simular o processo completo de uma encomenda, desde a escolha do produto pelo cliente até à sua entrega final por um drone.
- Implementar um mecanismo de gestão de recursos, neste caso os drones, de modo a garantir a sua utilização eficiente.
- Introduzir heterogeneidade nos agentes do tipo drone, através da definição de características como velocidade e capacidade de carga, obrigando as empresas a realizar uma escolha informada e estratégica dos drones a utilizar.
- Simular processos de tomada de decisão baseados em critérios técnicos, aproximando o comportamento do sistema a situações reais de gestão logística.
- Desenvolver um processo de negociação entre empresas para o empréstimo de drones quando existe escassez de recursos na central, promovendo a partilha e a colaboração.
- Analisar como agentes inteligentes podem resolver problemas de logística de forma descentralizada, sem necessidade de controlo central absoluto.
- Explorar a aplicabilidade de sistemas multiagente em contextos reais, como a distribuição de encomendas, gestão de frotas e comércio eletrónico.

Este programa pretende, assim, evidenciar o potencial dos sistemas de multiagentes inteligentes na resolução de problemas complexos, onde múltiplas entidades precisam de interagir e tomar decisões de forma autónoma, dinâmica e cooperativa.

3. Arquitetura do Sistema Multiagente

A arquitetura do sistema foi concebida para ser arquitetura multi-agente distribuída. O sistema organiza-se em torno de entidades autônomas que cooperam para garantir a fluidez e segurança do tráfego aéreo, onde a descentralização permite uma maior escalabilidade e tolerância a falhas.

3.1. Definição dos Agentes

Cada agente inclui (i) um estado interno (ex.: localização, carga, inventário), (ii) objetivo (ex.: completar entrega, negociar menor custo), (iii) comportamentos (*behaviours*) implementando tarefas e protocolos.

- **CentralAgent** — tem como objetivo vender drones, mediar alocações de drones quando necessário (negociação de drones) e monitorização do estado dos drones. *Behaviours: negotiate_behaviour, receiver_behaviour.*
- **CompanyAgent** — tem como objetivo gerir inventário da empresa, vender os seus produtos, oferecer serviços de entrega através dos seus drones. *Behaviours: package_manager_behaviour, receiver_behaviour.*
- **DroneAgent** — tem como objetivo executar missões de entrega de forma segura e eficiente. *Behaviours: delivery_behaviour, charging_behaviour, status_update_behaviour, receiver_behaviour.*
- **ClientAgent** — tem como objetivo apresentar uma interface ao utilizador, requisitar serviços de entrega e seguir o estado das compras. Este emite pedidos, avaliar propostas (preço), confirmar recepção e gera feedback. *Behaviours: buyer_behaviour, fetch_inventory_behaviour, receiver_behaviour.*

3.2. Arquitetura do Sistema

Em conformidade com os requisitos estabelecidos para o projeto, a modelação da arquitetura do sistema foi realizada utilizando a notação **Agent UML** (AUML). Esta norma permite a representação formal das particularidades dos sistemas multiagente, tais como a arquitetura interna dos agentes, os seus comportamentos (*behaviours*) e os protocolos de interação social.

3.2.1. Diagrama de Classes

O diagrama de classes abaixo detalha a estrutura estática da solução, evidenciando a organização interna de cada entidade do sistema. A representação ilustra as relações de herança e associação entre as classes principais (agentes) e as classes responsáveis pela lógica de execução (*behaviours*).

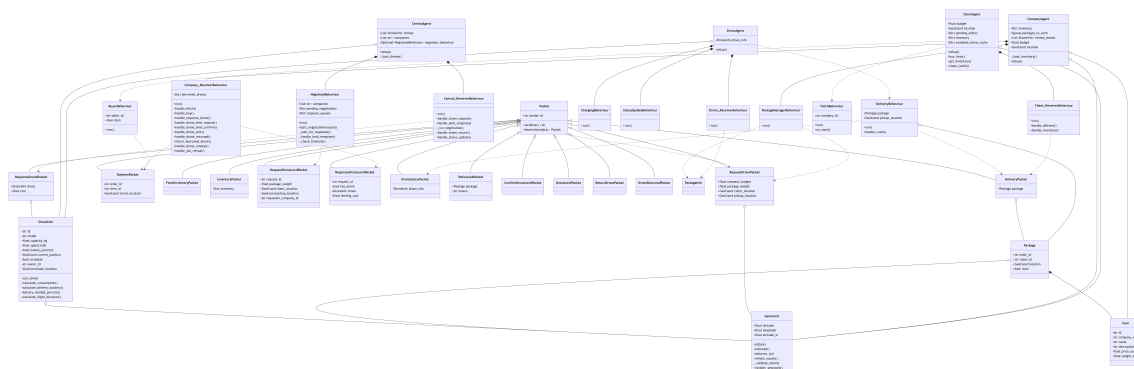


Figura 1: Diagrama de Classes

3.2.2. Diagrama de Sequência: Compra de Drone

O diagrama de sequência apresentado na figura seguinte modela o fluxo de interações despoletado por um pedido de encomenda, detalhando a lógica de decisão para a alocação de um veículo de transporte. O processo inicia-se com o envio do *BuyItemPacket* pelo Agente Cliente, o que leva o Agente Empresa a verificar prioritariamente a sua frota interna. Na ausência de drones disponíveis, o diagrama ilustra a escalabilidade do sistema através da solicitação de recursos ao Agente Central (via *RequestDronePacket*), culminando na aquisição de um drone ou, em caso de indisponibilidade total, no início de um processo de negociação.

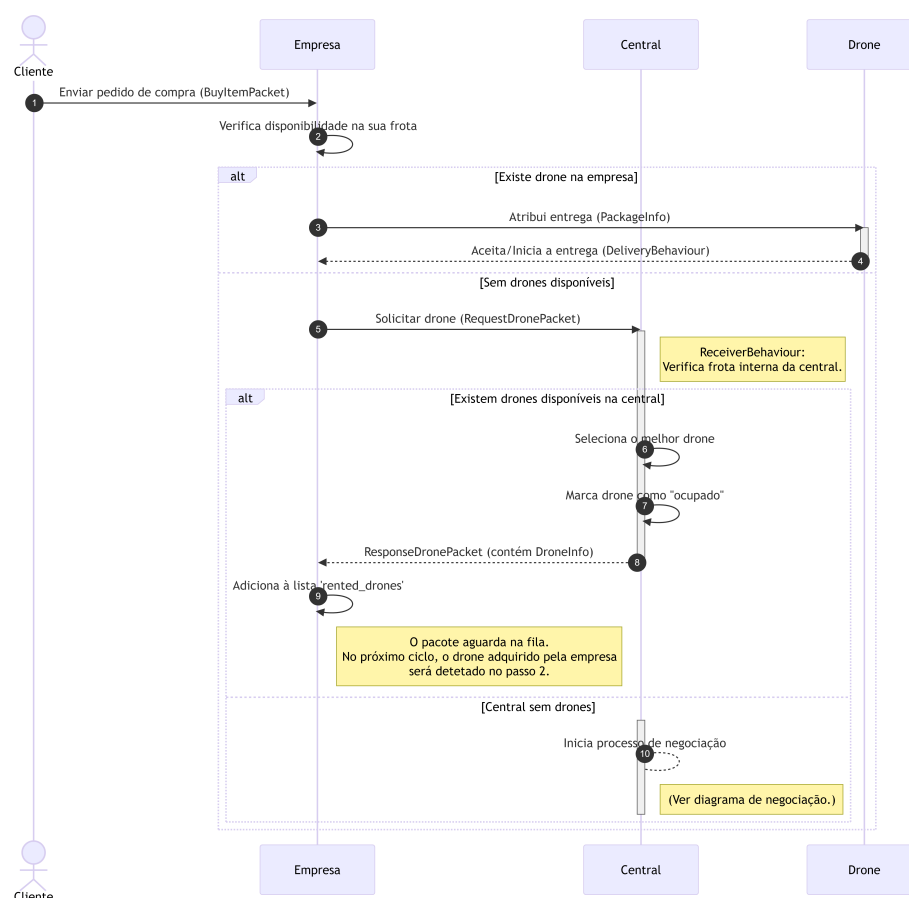


Figura 2: Diagrama de Sequência: Compra de Drone

3.2.3. Diagrama de Sequência: Compra de Item

O diagrama seguinte ilustra o ciclo de vida completo de uma transação, integrando a vertente comercial com a execução logística. O fluxo é desencadeado pelo envio de um *BuyItemPacket* pelo Agente Cliente, que leva o Agente Empresa a validar o stock e a processar o pedido através do seu *PackageManagerBehaviour*. O modelo destaca a gestão dinâmica da frota: na ausência de veículos locais, a Empresa solicita recursos externos ao Agente Central (via *RequestDronePacket*) antes de atribuir a missão através de um *DeliveryPacket*. Após a execução física da entrega pelo Agente Drone (incluindo recolha e *drop-off*), o processo encerra com a libertação do recurso, onde a Empresa decide, com base na propriedade do drone, se este fica disponível internamente ou se é devolvido a Central através de um *ReturnDronePacket*.

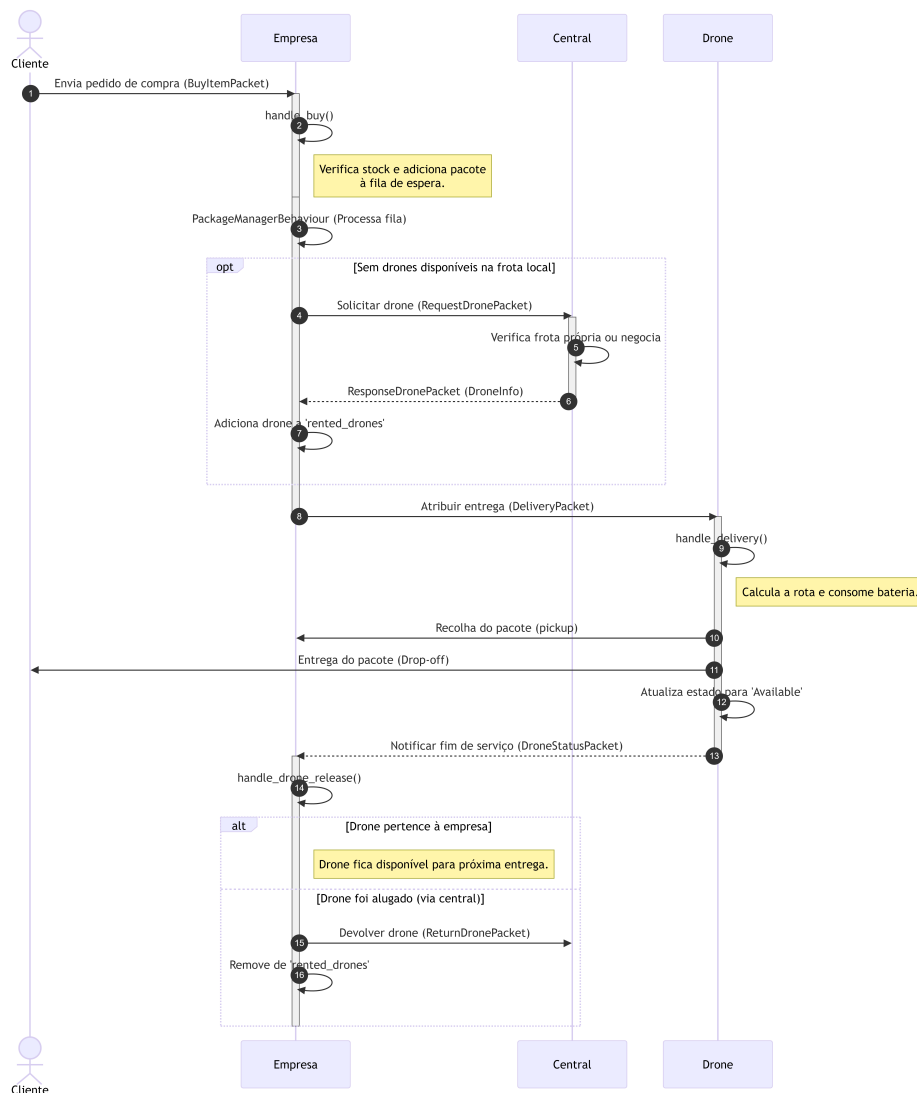


Figura 3: Diagrama de Sequência: Compra de Item

3.2.4. Diagrama de Sequência: Processo de Negociação

O diagrama abaixo modela o protocolo de cooperação inter-agentes acionado quando a frota da Central se encontra esgotada. O fluxo é iniciado pelo Agente Central, que atua

como mediador ao difundir um pedido de auxílio (*RequestDroneLendPacket*) em *broadcast* para as restantes empresas do sistema. O diagrama ilustra o mecanismo de partilha de recursos, onde as empresas fornecedoras avaliam a sua disponibilidade e calculam custos, submetendo propostas que permitem à Central selecionar a oferta mais eficiente para o sistema. O processo culmina na formalização do empréstimo, detalhando o ciclo de transferência temporária do *drone* para a empresa solicitante, a execução da entrega e a subsequente devolução do ativo à empresa proprietária.

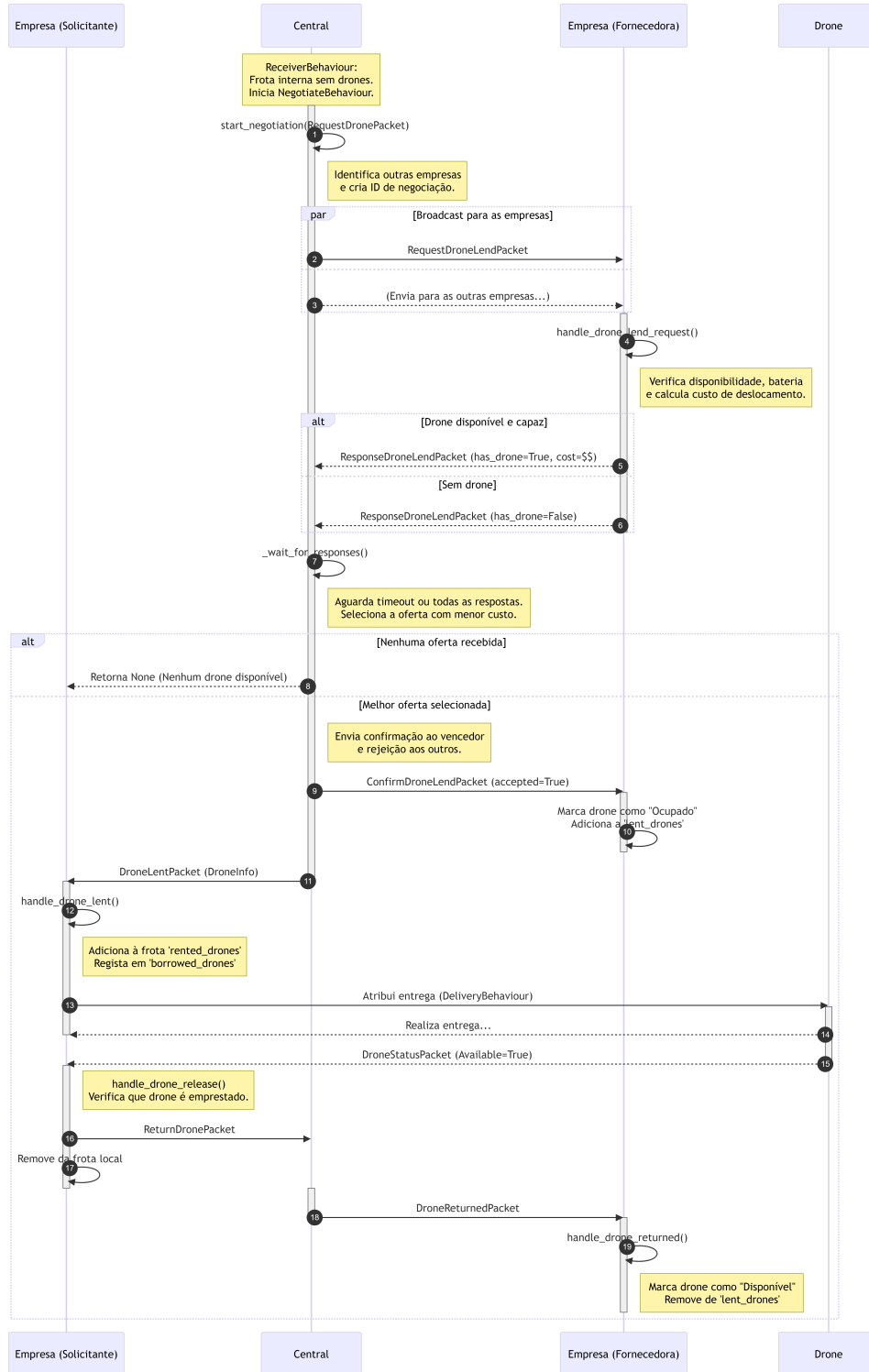


Figura 4: Diagrama de Sequência: Processo de Negociação

4. Comunicação e Protocolos

A comunicação é o pilar central deste sistema distribuído. Para garantir que os agentes interagem de forma inequívoca, definimos um protocolo de comunicação customizado.

4.1. Protocolos de Comunicação

O sistema implementa um protocolo de comunicação baseado em mensagens SPADE (Smart Python Agent Development Environment), onde cada mensagem transporta um pacote serializado em JSON utilizando a biblioteca *JSONpickle*. Este protocolo permite a serialização e deserialização automática de objetos Python complexos, mantendo a integridade dos tipos de dados e estruturas.

4.1.1. Estrutura Base de Comunicação

Todas as mensagens seguem a seguinte estrutura:

```
def new_message( packet: Packet, target_id: str ) -> Message:
    target_jid = get_agent_jid( target_id )
    msg = Message( to = target_id )
    msg.body = packet.serialize()
    return msg
```

O método *serialize()* da classe *Packet* converte o objeto para JSON, enquanto o método estático *deserialize()* reconstrói o objeto original a partir da string JSON recebida.

4.1.2. Tipos de Pacotes

O sistema define diversos tipos de pacotes especializados, cada um responsável por uma operação específica do protocolo:

4.1.2.1. Protocolo de Compra e Inventário

Pacote	Campos	Descrição
FetchInventoryPacket	sender_id	Solicitação do inventário atual de uma empresa.
InventoryPacket	sender_id, inventory	Resposta com o dicionário de itens disponíveis.
BuyItemPacket	sender_id, order_id, item_id, client_location	Pedido de compra de um item específico por um cliente.

Tabela 1: Mensagens do Protocolo de Compra e Inventário

4.1.2.2. Protocolo de Entrega

Pacote	Campos	Descrição
PackageInfo	sender_id, package, pickup_location	Dados sobre uma encomenda pronta para recolha.
DeliveryPacket	sender_id, package	Confirmação de que a entrega foi concluída com sucesso.
RefuseJobPacket	sender_id, package, reason	Notificação de recusa de um serviço com o respetivo motivo.

Tabela 2: Mensagens do Protocolo de Entrega

4.1.2.3. Protocolo de Gestão de Drones

Pacote	Campos	Descrição
RequestDronePacket	sender_id, company_budget, package_weight, client_location, pickup_location	Solicitação de drone com orçamento, peso da encomenda e localizações.
ResponseDronePacket	sender_id, drone, cost	Resposta com drone disponível (ou None) e custo estimado.
DroneStatusPacket	sender_id, drone_info	Atualização periódica de telemetria (bateria, posição, disponibilidade).

Tabela 3: Mensagens do Protocolo de Gestão de Drones

4.1.2.4. Protocolo de Empréstimo de Drones

Pacote	Campos	Descrição
RequestDroneLend-Packet	sender_id, request_id, package_weight, client_location, pickup_location, requester_company_id	Sondagem da Central às empresas para empréstimo externo de drone.
ResponseDroneLend-Packet	sender_id, request_id, has_drone, drone, lending_cost	Resposta da empresa com disponibilidade, drone oferecido e custo do empréstimo.
ConfirmDroneLend-Packet	sender_id, request_id, drone_id, requester_company_id, accepted	Confirmação da Central à empresa emprestadora sobre aceitação do empréstimo.
DroneLentPacket	sender_id, request_id, drone, lender_company_id, lending_cost	Notificação da Central à empresa requisitante com dados do drone emprestado.
ReturnDronePacket	sender_id, drone_id, lender_company_id	Devolução formal do drone pela empresa requisitante após o serviço.
DroneReturnedPacket	sender_id, drone_id	Confirmação da Central à empresa emprestadora de que o drone foi devolvido.

Tabela 4: Mensagens do Protocolo de Empréstimo de Drones

5. Aspectos adicionais do sistema

5.1. Drone

O drone representa o agente responsável pela execução física das entregas no sistema. Cada drone possui características próprias (`DroneInfo`), como a capacidade máxima de carga, a velocidade de deslocação e o nível de bateria disponível, que condicionam a sua utilização. Para além disso, cada drone tem uma posição geográfica associada, permitindo calcular distâncias, consumo energético e tempo de voo.

5.2. Item

O item corresponde ao produto que o cliente pretende adquirir. Cada item está associado a uma empresa, possui um nome, uma descrição, um preço e um peso. O peso do item é particularmente relevante, pois impacta o consumo de bateria dos drones durante a entrega e condiciona a escolha do drone mais adequado para transportar a encomenda. Assim, a gestão dos itens permite ao sistema integrar informações comerciais e logísticas, ligando o comércio eletrônico à operação de entrega.

5.3. Encomenda

A encomenda (`Package`) é a entidade que associa um cliente, um item e o destino da entrega. Cada encomenda contém o identificador do pedido, o cliente responsável, o item adquirido e a localização geográfica para entrega. Esta entidade é essencial para coordenar a interação entre clientes, empresas e drones, servindo como ponto de partida para o planeamento das rotas, cálculo do consumo de bateria e estimativa do tempo de entrega.

5.3.1. Coordenadas

No sistema desenvolvido, a localização geográfica dos drones, das empresas e dos clientes é representada através de coordenadas geográficas do tipo `GeoCoord` , baseadas no modelo *WGS-84*, utilizando latitude e longitude (e, opcionalmente, altitude). Estas coordenadas permitem calcular de forma realista a distância entre dois pontos no espaço, recorrendo à fórmula de *Haversine*, o que torna a simulação mais próxima de um cenário real.

A utilização de coordenadas geográficas é fundamental para determinar:

- A distância que um drone tem de percorrer até ao local de entrega;
- O consumo de bateria associado a cada deslocação;
- O tempo necessário para completar a entrega.

Desta forma, cada encomenda passa a ter uma componente espacial realista, obrigando o sistema a considerar a posição atual do drone e a localização do cliente antes de autorizar uma missão de entrega. Isto contribui para uma tomada de decisão mais informada por parte dos agentes, uma vez que não basta existir um drone disponível, sendo também necessário que este esteja suficientemente próximo e tenha autonomia para realizar o percurso.

5.3.2. Bateria

A bateria é um dos fatores mais críticos no funcionamento dos drones, sendo responsável por limitar a sua autonomia e a viabilidade das entregas. No sistema, o consumo de bateria é calculado com base em vários fatores:

- Um consumo base por quilómetro, mesmo quando o drone se desloca vazio;
- Um fator adicional dependente do peso da encomenda transportada;

- Um fator adicional associado à velocidade do drone;
- Uma margem de segurança, que garante que o drone mantém sempre uma percentagem mínima de bateria em reserva.

Antes de aceitar uma entrega, o drone verifica se possui bateria suficiente para realizar todo o percurso, incluindo a deslocação até ao destino, considerando o peso da encomenda, e mantendo a margem de segurança definida. Caso a bateria disponível seja inferior à necessária, o drone rejeita a missão.

Este mecanismo introduz uma restrição realista no sistema, simulando limitações físicas reais dos drones e obrigando as empresas a seleccionar não apenas um drone disponível, mas um drone energeticamente capaz de executar a tarefa com segurança.

5.3.3. Tempo de entrega

O tempo de entrega é diretamente influenciado pela distância a percorrer e pela velocidade do drone. No sistema, este valor é calculado através da relação entre a distância total do trajeto e a velocidade do drone, sendo convertido para segundos para facilitar a simulação temporal. Assim, o tempo de entrega pode ser simulado adicionando realismo ao programa.

5.4. Frontend

A interface de utilizador é implementada como um servidor HTTP leve no agente cliente e da central, usando **aihttp** para servir páginas HTML estáticas renderizadas com substituições simples. O frontend é composto por páginas HTML/CSS/JS (Single-page feel via reloads e fetch) que consomem *endpoints* JSON para operações dinâmicas.

5.4.1. Cliente

- *Home*: Lista as empresas, mostra saldo, localização e JID do cliente.
 - **GET /** : Exibe a página Home.
 - **GET /company/{company_id}** : Exibe a página da empresa.
 - **GET /purchases** : Exibe a página “Minhas Compras”.
- *Página da Empresa*: Mostra detalhes da empresa e os produtos disponíveis (após fetch de inventário).
 - **POST /api/fetch_inventory/{company_id}** : Envia um pedido ao *ClientAgent* para obter inventário da empresa.
 - **POST /api/buy_item** : Envia pedido de compra (payload JSON: company_id, item_id).
- *Minhas Compras*: Estatísticas e histórico de pedidos e inventário do cliente.
 - **GET /purchases** — Já referido.
 - **GET /api/client_info** : Endpoint JSON para obter estado do *ClientAgent* (JID, budget, location, pending_orders, cache_size, inventory_size).

5.4.2. Empresa

- *Dashboard*: Visão geral com estatísticas agregadas (inventário, pacotes, drones alugados) e resumo de todas as secções.
 - ▶ **GET /** : Exibe a página do Dashboard.
 - ▶ **GET /inventory**: Exibe a página de Inventário.
 - ▶ **GET /packages**: Exibe a página de Gestão de Pacotes.
 - ▶ **GET /drones**: Exibe a página de Frota de Drones.
- Página de Inventário: Lista completa dos produtos disponíveis na empresa com detalhes.
 - ▶ **GET /api/inventory**: Endpoint JSON que retorna o inventário (id, name, price, weight, description, company_id, category).
- Página de Pacotes: Gestão completa do ciclo de vida dos pacotes com estados.
 - ▶ **GET /api/packages**: Endpoint JSON que retorna todos os pacotes (order_id, client_id, destination, weight, item_name, item_id, status).
 - ▶ Estados: **Na Fila** (amarelo), **Em Entrega** (azul), **Entregue** (verde).
- Página de Drones: Monitorização da frota de drones alugados com informações em tempo real.
 - ▶ **GET /api/drones**: Endpoint JSON que retorna a frota (id, model, capacity_kg, speed_kmh, battery_level, available, current_position).
- Estatísticas Gerais: Dados agregados sobre a operação da empresa.
 - ▶ **GET /api/stats**: Estatísticas agregadas (company_id, budget, location, inventory_items, total_quantity, total_value, rented_drones).

5.4.3. Central

- *Dashboard*: Painel administrativo com estatísticas da frota, empresas registadas e negociações ativas.
 - ▶ **GET /** : Exibe a página index (dashboard).
 - ▶ **GET /api/stats** : Estatísticas agregadas (drones totais, disponíveis, capacidade total, bateria média, test_mode).
 - ▶ **GET /api/drones** : Lista de drones com estado (id, modelo, capacidade, velocidade, bateria, available).
 - ▶ **GET /api/companies** : Lista de empresas registadas.
 - ▶ **GET /api/negotiations** : Estado das negociações activas.
 - ▶ **POST /api/toggle-drone** : Toggle (available/busy) para um drone (payload JSON: drone_id).
 - ▶ **POST /api/test-mode** : Activa/desactiva modo de teste (payload JSON: enabled).

6. Análise de Resultados e Crítica

6.1. Sumário de Resultados

O sistema desenvolvido suporta todos os objetivos e funcionalidades pretendidas quando o mesmo foi planeado. O sistema suporta a compra de drones, a atribuição de encomendas realizada pelos mesmos, permitindo também se necessário iniciar um processo de negociação para aluguer de drones. O sistema dispõe de testes, que foram passados com sucesso, e ainda de várias interfaces para interagir com o mesmo manualmente, permitindo realizar várias das funcionalidades, como observar o estado dos drones e dos pedidos feitos. Como já foi referido, existem testes automáticos, que funcionam como simulações para validar mais facilmente o sistema, estes foram realizados e executados para validar o sistema. Concluindo, o sistema apresentado é adequado para a gestão de entregas descentralizada utilizando drones, obtendo sucesso nas funcionalidades esperadas.

6.2. Análise Crítica

Do ponto de vista técnico, o modelo de comunicação baseado em “*Packets*” é um dos aspetos mais positivos, pois cria uma abstração coerente para troca de mensagens e permite que o sistema seja facilmente expandido com novos tipos de interação. Paralelamente, a modelação do domínio é credível, já que inclui elementos relevantes como estado do drone, coordenadas geográficas e encomendas, complementados por uma simulação física consistente onde o consumo de bateria depende de distância, peso e velocidade, aproximando o sistema de condições mais realistas.

Entre os pontos mais fortes destaca-se também o protocolo de negociação distribuída para empréstimo de drones entre empresas, que representa bem um problema típico de coordenação e alocação dinâmica de recursos. A inclusão de testes de integração e de uma interface web de monitorização reforça a utilidade prática do projeto e facilita validação do fluxo completo, além de tornar a demonstração mais clara e visível.

No entanto, o projeto ainda apresenta limitações importantes em termos de robustez e realismo. A ausência de persistência de dados impede histórico de entregas, recuperação após falhas e auditoria do sistema. Além disso, a segurança é fraca, incluindo credenciais *hardcoded* e ausência de autenticação e autorização na API, o que compromete a confiabilidade e abre margem para comportamentos indevidos. A gestão de erros é limitada, com exceções tratadas de forma genérica e, por vezes, ignoradas, e o sistema não possui mecanismos essenciais para lidar com falhas de rede, como *timeouts*, *retries* e deteção de agentes indisponíveis, tornando-o vulnerável em cenários distribuídos mais próximos da realidade.

Para aumentar realismo e valor científico, o sistema pode evoluir com algoritmos de otimização de rotas e escalonamento de entregas, prioridades e SLAs, monitorização em tempo real e simulação de fatores externos como condições meteorológicas e zonas de exclusão aérea. Estas e outras extensões serão discutidas no próximo tópico.

7. Recomendações e Trabalho Futuro

Embora o sistema atual demonstre as funcionalidades essenciais de um SMA, várias extensões podem enriquecer a complexidade e o realismo da solução:

7.1. Ambiente e Contexto

- **Simulação de Clima:** Modelar condições meteorológicas (vento, chuva) que afetem o consumo energético e a velocidade, obrigando os agentes a raciocinar sobre incerteza.
- **Zonas de Restrição Aérea:** Implementar *no-fly zones* dinâmicas que requeiram o replaneamento de rotas em tempo real por parte dos drones.
- **Evitação de Colisões:** Desenvolver protocolos de evitação descentralizados para cenários onde múltiplos drones partilham trajetórias sobrepostas em baixa altitude.

7.2. Gestão e Logística de Inventário

- **Controlo de Stock Dinâmico:** Implementar uma lógica de inventário real em cada Agente Empresa. Isto obrigaria o sistema de negociação a considerar não só a disponibilidade de drones, mas também a existência do produto em armazéns de diferentes empresas, possibilitando compras multi-empresa.
- **Reabastecimento Autónomo:** Criar comportamentos onde as empresas negociem entre si ou com agentes fornecedores o reabastecimento automático de produtos quando o stock atinge um nível crítico.

8. Conclusão

O sistema desenvolvido demonstra com sucesso a aplicação de tecnologias de agentes inteligentes na resolução de problemas complexos de logística e coordenação. Através da framework SPADE, foi possível implementar uma arquitetura distribuída onde múltiplos agentes autónomos cooperam para realizar entregas de encomendas via drones.

Os objetivos principais foram alcançados:

- **Autonomia e Reatividade:** Cada agente toma decisões independentes baseadas no seu estado interno e nas mensagens recebidas. Os drones decidem autonomamente aceitar ou recusar trabalhos com base em cálculos de bateria e capacidade, demonstrando comportamento reativo às condições do ambiente.

- **Coordenação e Negociação:** O protocolo de empréstimo de drones implementado demonstra um mecanismo sofisticado de negociação multiagente, onde empresas concorrentes cooperam através da mediação do agente central. Este protocolo ilustra como agentes podem colaborar mesmo em ambientes competitivos, otimizando recursos do sistema global.
- **Escalabilidade:** A arquitetura distribuída permite adicionar novos agentes (empresas, drones, clientes) sem modificação do código base. Esta característica é fundamental em sistemas multiagente reais, onde a elasticidade do sistema é um requisito.
- **Comunicação Estruturada:** O protocolo de comunicação baseado em pacotes serializados em JSON garante a integridade dos dados e a facilidade de extensibilidade do sistema, cumprindo o requisito de utilizar objetos em vez de strings simples.

A validação através de casos de uso completos — desde a intenção de compra do cliente até à entrega final e devolução do drone emprestado — confirma a robustez da implementação. O sistema responde adequadamente tanto a cenários nominais como a situações de exceção, como bateria insuficiente ou recusa de tarefas por falta de capacidade técnica.

8.1. Reflexão Final

A realização deste projeto permitiu consolidar conhecimentos sobre sistemas distribuídos e inteligência artificial descentralizada. A utilização do SPADE revelou-se uma escolha acertada pela sua flexibilidade e conformidade com as normas FIPA, proporcionando uma base sólida para a resolução de problemas de logística moderna.